

Tennis World Project Checkpoint

Purpose of this checkpoint

This checkpoint summarizes the Tennis World project from the beginning up to the current local testing state.

Use this document to continue work in a new ChatGPT window without losing context.

Current status:

- ✓ Phase 0-5 development is implemented
- ✓ Local Postgres works
- ✓ API works locally
- ✓ Web app works locally
- ✓ Web is connected to the live API, not mock data
- ✓ Prisma migrations and seed completed successfully
- ✓ All non-Stripe harnesses are green
- ✓ Stripe is intentionally skipped for now because there is no Stripe account yet
- ➡ Next step: manual browser smoke-test without Stripe

1. Project Overview

Project name:

Tennis World

Local project path:

D:\work\tennis

The project is a monorepo for a premium tennis court discovery product.

The product idea:

- public discovery of beautiful tennis courts;
- public court detail pages;
- collections;
- journal/articles;
- consultation flow;
- user accounts;

- saved courts;
 - user-created collections;
 - locked premium courts/features;
 - premium exact-location access;
 - future Stripe payments and entitlements;
 - future admin app.
-

2. Monorepo Structure

The repository contains:

```
apps/api      – NestJS API
apps/web      – Next.js frontend
apps/admin    – admin app placeholder / future admin
packages/*    – shared contracts, mock data, config, etc.
```

Important current apps:

```
apps/api
```

NestJS 10 API with Prisma/Postgres.

```
apps/web
```

Next.js App Router frontend with TypeScript and Tailwind.

```
apps/admin
```

Admin placeholder for later phases.

3. Main Technology Stack

Frontend

```
Next.js App Router
TypeScript
Tailwind CSS
Server Components
Client islands where needed
```

API

```
NestJS 10  
Prisma 6.19.3  
Postgres 16  
Cookie-based auth  
Magic link login
```

Database

```
PostgreSQL  
Docker Compose  
Prisma migrations  
Prisma seed from @tennis/mock-data
```

Package manager

```
pnpm
```

Important architectural constraint

The web app can run in two modes:

```
NEXT_PUBLIC_DATA_SOURCE=mock
```

or:

```
NEXT_PUBLIC_DATA_SOURCE=api
```

For integration testing, it must be:

```
NEXT_PUBLIC_DATA_SOURCE=api
```

4. Important Architecture Rules

These constraints are important and should be preserved.

No API routes inside Next.js web app

There should be no API implementation inside:

```
apps/web/app/api
```

The API must live in:

```
apps/api
```

Web UI must not import mock data directly

The web UI should not import:

```
@tennis/mock-data
```

directly.

Mock data can be used behind repository implementations, but UI components/pages should use repository boundaries.

Repository boundary

Only appropriate server/page-level code should import web repositories.

Do not casually import repository internals into random components.

Public API must not expose exact coordinates

Public court data must not expose exact DB coordinates:

```
lat  
lng
```

Public data can expose approximate/map coordinates only.

Exact coordinates are premium/protected.

Exact location is protected

Exact court coordinates are available only through protected entitlement-based endpoint:

```
GET /v1/me/courts/:slug/exact-location
```

5. Development Phase Summary

Phase 0

Initial monorepo and project foundations.

Verified:

```
apps/api health only
apps/admin placeholder
packages/contracts
packages/mock-data
```

Core shared data/contracts included:

```
Court
CourtSummary
Collection
Article
Consultation
User
Entitlement
```

Mock data included:

```
12 courts
6 collections
3 articles
countries
regions
images
```

Important detail:

Exact coordinates are separate from public map coordinates.

Feature 2

Repository factory and data-source boundary.

The web app can switch between:

```
NEXT_PUBLIC_DATA_SOURCE=mock
```

and:

```
NEXT_PUBLIC_DATA_SOURCE=api
```

Feature 3

Frontend app shell and base UI were created.

Included:

```
AppShell  
AppHeader  
BottomNavigation  
PageContainer  
Button  
SectionHeader  
Badge
```

Tailwind theme foundation was added.

Feature 4

Court card component was created.

CourtCard includes:

```
image  
name  
location  
surface  
access  
scenic badge  
locked/open/featured state  
saved visual state  
href
```

Home Page

Home page includes:

```
Hero
"The World of Tennis"
"Where the game meets the extraordinary."
stats line
featured courts carousel
```

6. Later Features Before Phase 5

The project continued with:

```
Court detail pages
Collections
Journal/articles
Consultation
Saved courts
Profile
Auth foundation
Magic link login
Protected user areas
API repositories
API parity checks
```

Important auth behavior:

- private pages redirect logged-out visitors to signin;
- `/saved` redirects to signin when logged out;
- profile is protected;
- saved courts are protected;
- user collections are protected;
- public discovery pages remain accessible.

Example observed behavior:

```
GET /saved 307
GET /signin?redirectTo=%2Fsaved 200
```

This is correct.

7. Phase 5 Summary — Payments and Entitlements

Phase 5 implemented payments/entitlement groundwork.

Stripe is implemented in code but not tested end-to-end yet because there is no Stripe account.

Feature 61 — Schema and contract groundwork

Added/expanded entitlement-related fields.

Entitlement fields include:

```
providerCustomerId
subscriptionId
purchaseId
startsAt
revokedAt
revocationReason
```

User billing-related field:

```
stripeCustomerId
```

Webhook idempotency model:

```
ProcessedWebhookEvent
```

Billing/exact-location DTOs were added.

Feature 62 — Effective entitlement derivation

Implemented service that derives effective membership from entitlement records.

Endpoint:

```
GET /v1/me
```

returns membership state based on entitlements.

Feature 63 — Protected exact location endpoint

Added protected exact-location endpoint:

```
GET /v1/me/courts/:slug/exact-location
```

This endpoint must only return exact coordinates if the user has proper entitlement.

Feature 64 — Web exact-location wiring

Frontend was wired to support exact-location premium behavior.

Expected behavior:

- free users should not see premium exact-location data;
 - premium users should be able to access exact location;
 - UI should handle locked/free states safely.
-

Feature 65 — Stripe checkout and portal endpoints

Billing endpoints added:

```
POST /v1/billing/checkout  
POST /v1/billing/portal
```

These require Stripe env configuration to function fully.

Feature 66 — Stripe webhook fulfillment

Webhook endpoint added:

```
POST /v1/webhooks/stripe
```

It processes Stripe events and creates/updates entitlements.

Webhook processing is idempotent via `ProcessedWebhookEvent`.

Feature 67 — Web paywall / portal / return wiring

Frontend billing flow wired for:

```
checkout
billing return
profile membership display
portal return
paywall CTA
```

Feature 68 — Optional Stripe E2E smoke

Optional Stripe smoke harness added.

Currently skipped because Stripe account and CLI are not configured.

Feature 69 — Billing rate limiting

Billing-related endpoints have rate-limit verification.

The harness passed locally.

Feature 70 — Phase 5 completion summary

Phase 5 was completed at implementation level.

Current limitation:

```
Stripe has not been manually tested end-to-end yet.
```

8. Current Local Machine Issue — Windows Reserved Ports

The local Windows machine has excluded/reserved TCP port ranges.

Command used:

```
netsh interface ipv4 show excludedportrange protocol=tcp
```

Important returned range:

2970–3069

This range includes:

3000
3001
3002
3003

Because of that, default dev ports failed.

API failed on default-ish ports:

```
Error: listen EACCES: permission denied 0.0.0.0:3001
Error: listen EACCES: permission denied 0.0.0.0:3003
```

Web failed on default Next port:

```
Error: listen EACCES: permission denied 0.0.0.0:3000
```

Postgres also had issues with common ports:

5432
5433

Solution: use high local ports.

Current working local ports:

```
Postgres: localhost:15432
Web:      127.0.0.1:18000
API:      127.0.0.1:18001
```

9. Current Working Docker/Postgres Setup

Postgres runs in Docker Compose.

Expected service:

```
postgres
```

Expected container name:

```
tennis_world_pg
```

Expected DB credentials:

```
POSTGRES_DB=tennis_world  
POSTGRES_USER=tennis  
POSTGRES_PASSWORD=tennis
```

Expected exposed port:

```
Host:      15432  
Container: 5432
```

Working `docker-compose.yml` should contain something like:

```
services:  
  postgres:  
    container_name: tennis_world_pg  
    image: postgres:16  
    environment:  
      POSTGRES_DB: tennis_world  
      POSTGRES_USER: tennis  
      POSTGRES_PASSWORD: tennis  
    ports:  
      - "127.0.0.1:15432:5432"  
    volumes:  
      - tennis_pg_data:/var/lib/postgresql/data  
    restart: unless-stopped  
  
volumes:  
  tennis_pg_data:
```

This form may also work:

```
ports:  
  - "15432:5432"
```

The important part is that Prisma connects to:

```
localhost:15432
```

10. Current API Environment

Working file:

```
apps/api/.env
```

Current working content should be equivalent to:

```
DATABASE_URL="postgresql://tennis:tennis@localhost:15432/tennis_world?
schema=public"

PORT=18001
HOST=127.0.0.1

JWT_SECRET=change-me
AUTH_COOKIE_NAME=tennis_session
AUTH_TOKEN_TTL_SECONDS=3600
MAGIC_LINK_TTL_MINUTES=15

WEB_APP_URL=http://127.0.0.1:18000
API_CORS_ORIGINS=http://127.0.0.1:18000
AUTH_COOKIE_SECURE=false
MAGIC_LINK_DEV_LOG=true
```

Important notes:

- The project uses `PORT` and `HOST`, not necessarily `API_PORT` and `API_HOST`.
- API listens on:

```
http://127.0.0.1:18001/v1
```

Observed successful API startup:

```
[api] listening on http://127.0.0.1:18001/v1/health
```

API health/courts check:

```
curl.exe http://127.0.0.1:18001/v1/courts
```

11. Current Web Environment

Important: Next.js reads:

```
apps/web/.env.local
```

Not:

```
apps/web/.env.example
```

There was a temporary mistake where `.env.example` was edited, but `.env.local` is the file that matters.

Working file:

```
apps/web/.env.local
```

Current working content:

```
NEXT_PUBLIC_DATA_SOURCE=api  
NEXT_PUBLIC_API_BASE_URL=http://127.0.0.1:18001/v1  
API_BASE_URL=http://127.0.0.1:18001/v1
```

Important correction:

This is wrong:

```
NEXT_PUBLIC_API_BASE_URL=http://127.0.0.1:18000/v1  
API_BASE_URL=http://127.0.0.1:18000/v1
```

Because `18000` is the web port.

Correct:

```
NEXT_PUBLIC_API_BASE_URL=http://127.0.0.1:18001/v1  
API_BASE_URL=http://127.0.0.1:18001/v1
```

Because `18001` is the API port.

12. Current Local Startup Procedure

Use three terminal windows.

Terminal 1 — Postgres

From repo root:

```
cd D:\work\tennis  
docker compose up -d
```

Check:

```
docker compose ps
```

Check host port:

```
Test-NetConnection localhost -Port 15432
```

Expected:

```
TcpTestSucceeded : True
```

Terminal 2 — API

From repo root:

```
cd D:\work\tennis  
pnpm --filter @tennis/api dev
```

Expected successful output:

```
[api] listening on http://127.0.0.1:18001/v1/health
```

Check:

```
curl.exe http://127.0.0.1:18001/v1/courts
```

Terminal 3 — Web

From web app folder:

```
cd D:\work\tennis\apps\web
pnpm dev -- -p 18000 -H 127.0.0.1
```

Or from repo root:

```
cd D:\work\tennis
pnpm --filter @tennis/web dev -- -p 18000 -H 127.0.0.1
```

Open:

```
http://127.0.0.1:18000
```

Expected web startup output:

```
- Environments: .env.local
✓ Starting...
✓ Ready
```

13. Prisma Migration and Seed Status

Migrations were run successfully:

```
pnpm --filter @tennis/api prisma:migrate:deploy
```

Successful output:

```
Environment variables loaded from .env
Prisma schema loaded from prisma/schema.prisma
Datasource "db": PostgreSQL database "tennis_world", schema "public" at
"localhost:15432"

4 migrations found in prisma/migrations

No pending migrations to apply.
```

Seed was run successfully:


```
pnpm --filter @tennis/api db:seed
```

Successful output:

```
Environment variables loaded from .env
Running seed command `tsx prisma/seed.ts` ...
Seeding Postgres from @tennis/mock-data ...
Seed complete. Row counts:
```

countries	11
regions	12
courts	12
courtImages	30
collections	6
collectionMemberships	15
articles	3

```
The seed command has been executed.
```

14. Completed Automated Checks

The first group of build/lint/type checks passed.

Then all non-Stripe harnesses passed.

Before running harnesses, the following env values were set in PowerShell:

```
$env:NEXT_PUBLIC_API_BASE_URL="http://127.0.0.1:18001/v1"
$env:API_BASE_URL="http://127.0.0.1:18001/v1"
$env:WEB_APP_URL="http://127.0.0.1:18000"
```

Harness commands:

```
pnpm verify:api-parity
pnpm --filter @tennis/api verify:effective-entitlement
pnpm --filter @tennis/api verify:exact-location
pnpm --filter @tennis/api verify:billing-rate-limit
pnpm --filter @tennis/web verify:web-billing
pnpm --filter @tennis/web verify:web-exact-location
```

Current status:

✓ All tests are green

Meaning:

- ✓ API parity checks passed
- ✓ Effective entitlement logic passed
- ✓ Exact-location access checks passed
- ✓ Billing rate-limit checks passed
- ✓ Web billing checks passed
- ✓ Web exact-location checks passed

15. Current Browser Behavior

The web app starts and serves pages.

Observed Next logs:

```
GET /saved 307
GET /signin?redirectTo=%2Fsaved 200
GET / 200
GET /courts/grand-hotel-tremezzo 200
```

This is correct.

Meaning:

```
/saved redirects to /signin when logged out
/signin renders successfully
/ renders successfully
/courts/grand-hotel-tremezzo renders successfully
```

16. Known Non-Blocking Issue — Unsplash 404 Images

Current web logs include errors like:

```
upstream image response failed for https://images.unsplash.com/
photo-1539020140153-e479b8c5b1bc?w=1400&q=80&auto=format&fit=crop 404
```

```
upstream image response failed for https://images.unsplash.com/
photo-1530915534234-66dcdb7e2f5d?w=1400&q=80&auto=format&fit=crop 404
```

This is not blocking.

Meaning:

```
Some external Unsplash image URLs in mock/seed data are dead.
```

The app still works.

Later fix:

```
Replace dead image URLs in seed/mock data
```

or use stable local/static images.

17. Stripe Status

Stripe is intentionally skipped for now.

Reason:

```
No Stripe account yet
Stripe CLI is not installed
```

When trying:

```
stripe listen --forward-to http://127.0.0.1:18001/v1/webhooks/stripe
```

PowerShell returned:

```
stripe : The term 'stripe' is not recognized as the name of a cmdlet,
function, script file, or operable program.
```

This simply means Stripe CLI is not installed or not in PATH.

Do not continue Stripe work until there is a Stripe account.

18. Stripe Work To Do Later

When a Stripe account exists, do this later.

Install Stripe CLI

Possible options:

- Scoop
- direct `stripe.exe`
- add Stripe CLI folder to PATH

Then:

```
stripe login
```

Run listener:

```
stripe listen --forward-to http://127.0.0.1:18001/v1/webhooks/stripe
```

Stripe will output:

```
whsec_...
```

Add to `apps/api/.env`:

```
STRIPE_WEBHOOK_SECRET=whsec_...
```

Also configure:

```
STRIPE_SECRET_KEY=sk_test_...  
STRIPE_PRICE_LIFETIME=price_...  
STRIPE_SUCCESS_URL=http://127.0.0.1:18000/billing/return  
STRIPE_CANCEL_URL=http://127.0.0.1:18000/profile?checkout=cancelled  
STRIPE_PORTAL_RETURN_URL=http://127.0.0.1:18000/profile
```

Then restart API.

Use Stripe test card:

```
4242 4242 4242 4242
```

Expected later Stripe E2E checks:

```
Checkout creates session
Webhook receives event
Entitlement is created
/v1/me shows premium/lifetime
Exact location becomes available
Profile shows premium state
Billing return page works
```

19. Next Step After This Checkpoint

Continue with:

```
Manual browser smoke-test without Stripe
```

Start from:

```
http://127.0.0.1:18000
```

20. Manual Smoke-Test Plan Without Stripe

20.1 Guest Home Page

Open:

```
http://127.0.0.1:18000
```

Check:

```
Home page loads
Hero is visible
Featured courts are visible
No fatal error
Dead Unsplash image warnings are acceptable for now
```

20.2 Guest Court Detail

Open a known seeded court:

```
http://127.0.0.1:18000/courts/grand-hotel-tremezzo
```

Check:

```
Court detail page loads  
Court name is visible  
Public information is visible  
No exact premium coordinates are exposed  
Locked/premium CTA behavior is visible where expected
```

20.3 Saved Page Logged-Out Redirect

Open:

```
http://127.0.0.1:18000/saved
```

Expected:

```
Redirects to /signin?redirectTo=%2Fsaved
```

This behavior was already observed and is correct.

20.4 Signin Page

Open:

```
http://127.0.0.1:18000/signin
```

Check:

```
Signin form loads  
Email field works  
Magic link request can be submitted
```

20.5 Magic Link Login

Make sure API env has:

```
MAGIC_LINK_DEV_LOG=true  
WEB_APP_URL=http://127.0.0.1:18000
```

On signin page, enter:

```
test@example.com
```

Expected:

```
API console prints a magic link
```

The link should look like:

```
http://127.0.0.1:18000/verify?token=...
```

Open that link in the browser.

Expected:

```
User becomes logged in  
Auth cookie is set  
Redirect works
```

20.6 Profile After Login

Open:

```
http://127.0.0.1:18000/profile
```

Expected:

```
Profile loads  
User is logged in  
Membership is free  
No premium entitlement shown  
Billing/upgrade UI exists but Stripe is not configured
```

20.7 Saved After Login

Open:

```
http://127.0.0.1:18000/saved
```

Expected:

```
Saved page loads  
No redirect to signin  
Empty or seeded user-specific saved state is handled safely
```

20.8 Free User Premium Lock Behavior

As a logged-in free user, check locked/premium court behavior.

Expected:

```
Locked court remains locked  
Paywall/upgrade CTA appears  
Exact location is not available  
Get Directions should not expose premium coordinates
```

20.9 Checkout Without Stripe

Because Stripe env is empty/missing, clicking checkout/upgrade should not crash the app.

Expected:

```
Safe controlled error  
No white screen  
No server crash  
No unhandled fatal exception
```

A controlled error like this is acceptable:

```
Billing is not configured
```


20.10 Public API Safety Check

Check public API:

```
curl.exe http://127.0.0.1:18001/v1/courts
```

Important:

```
Public response must not expose exact DB lat/lng
```

If there are approximate map coordinates, that is expected.

Exact coordinates should only be available through:

```
GET /v1/me/courts/:slug/exact-location
```

and only for entitled users.

21. Current Git/Commit Reminder

Check current changes:

```
git status
```

Likely modified files may include:

```
docker-compose.yml  
apps/api/src/main.ts  
apps/api/.env.example  
apps/web/package.json
```

Do not commit local secret env files:

```
apps/api/.env  
apps/web/.env.local
```

Potential commit:

```
git add docker-compose.yml apps/api/src/main.ts apps/api/.env.example apps/web/package.json
git commit -m "Fix local development ports"
```

If some files were not changed, do not add them.

Always inspect:

```
git status
```

before committing.

22. Important Troubleshooting Notes

If Prisma cannot connect

Check Docker:

```
docker compose ps
```

Check port:

```
Test-NetConnection localhost -Port 15432
```

Check API env:

```
DATABASE_URL="postgresql://tennis:tennis@localhost:15432/tennis_world?
schema=public"
```

If API fails with EACCES on 3001/3003

Do not use 3000–3069 on this Windows machine.

Use:

```
PORT=18001
HOST=127.0.0.1
```

Start API:

```
pnpm --filter @tennis/api dev
```

If API fails with EADDRINUSE on 18001

It means another process is already using the port.

Check:

```
netstat -ano | findstr :18001
```

Find process:

```
tasklist /FI "PID eq <PID>"
```

Kill if needed:

```
taskkill /PID <PID> /F
```

But if API already responds, do not start a second API instance.

Check:

```
curl.exe http://127.0.0.1:18001/v1/courts
```

If Web fails with EACCES on 3000

Do not run:

```
pnpm dev
```

unless the package script has been changed.

Run:

```
pnpm dev -- -p 18000 -H 127.0.0.1
```

or update `apps/web/package.json` :

```
"dev": "next dev -p 18000 -H 127.0.0.1"
```

If Web does not use API

Check actual file:

```
apps/web/.env.local
```

It must contain:

```
NEXT_PUBLIC_DATA_SOURCE=api
NEXT_PUBLIC_API_BASE_URL=http://127.0.0.1:18001/v1
API_BASE_URL=http://127.0.0.1:18001/v1
```

Restart Next after changing env.

If CORS/Auth fails

Check API env:

```
WEB_APP_URL=http://127.0.0.1:18000
API_CORS_ORIGINS=http://127.0.0.1:18000
AUTH_COOKIE_SECURE=false
```

Restart API after changing env.

23. Current Final State

As of this checkpoint:

- ✓ Project is locally running without Stripe
- ✓ Database is seeded
- ✓ API is reachable
- ✓ Web is reachable
- ✓ Web is using API mode
- ✓ Automated non-Stripe harnesses are green
- ✓ Browser smoke-test is the next task

Continue from here with:

Manual browser smoke-test without Stripe